UNIVERSITY OF WATERLOO

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

ECE-124 LAB MANUAL

V5.0

LAB 4

Course: ECE-124 Digital Circuits and Systems (with Verilog)

1 LAB4: VERILOG for Sequential Circuits – Sequential Logic & State-Machines

Lab4 will be the first one that uses Sequential Logic. Background information on sequential logic will be briefly covered to suit the Lab4 requirements. Design elements from previous labs will be used for Lab4. Both Moore and Mealy State Machine designs will be in the design goals for this lab.

1.1 Lab4 Intended Learning Outcomes

By the end of this lab, students should be able to:

1) **UNDERSTAND** how to avoid META-STABILITY in Sequential Logic
2) **UNDERSTAND** State Machine design concepts, including Moore or Mealy forms.
3) **IMPLEMENT a** State Machine design

1.2 Lab4 Outline

Attendance will be taken.

The following new topics will be presented:

1. Brief Review on Sequential Logic Processing from Lab 3
2. What is Meta-Stability?
3. New VERILOG Component – What are State Machines?
4. Lab4 Project Setup
5. Project Brief for Lab4

1.3 Lab4 Activities

1.3.1 Recall from Lab3:

Two different Statement Domains in HDL's are the Concurrent and Sequential Statement domains.

In the Concurrent Statement domain, the Structural and Dataflow styles of Verilog coding were used in earlier labs to implement Combinational logic only.

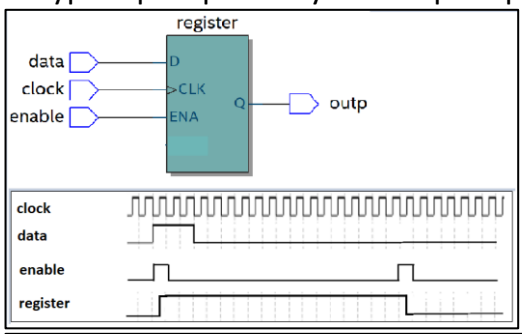
Beginning in Lab3, a Behavioral VERILOG style was introduced. This style can only be implemented in the Sequential Statement domain and it employs a special construct, called the Always Procedural block. The Always Procedural blocks can be used to form Combinational Logic or Sequential Logic.

REVISE The combinational logic, created in an Always Procedural block, was a Tester function and it could use Behavioural style IF statements etc. to test results of an upstream Magnitude Comparator function.

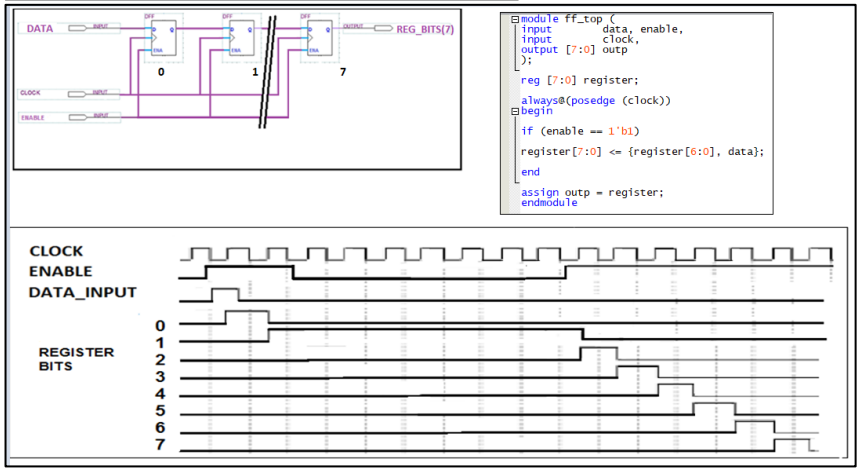
Also, in Lab3, a bidirectional Up/Down Counter was created. The comparison function was implemented in two levels. This Magnitude Comparator function was used in an Energy Monitoring Controller to determine current and desired temperature differences.

1.3.2 Recall Sequential Logic

Sequential logic, like was covered in Lab3, uses "storage elements" like latches and Flip-Flops. There is an input that is captured by the clock signal into the storage element. The captured signal is then visible at the output of the storage element. For this course the storage elements are typically just of the D-Type flip-flop variety. The Flip-Flops can be controlled by other signals such as reset, Clock_Enable.



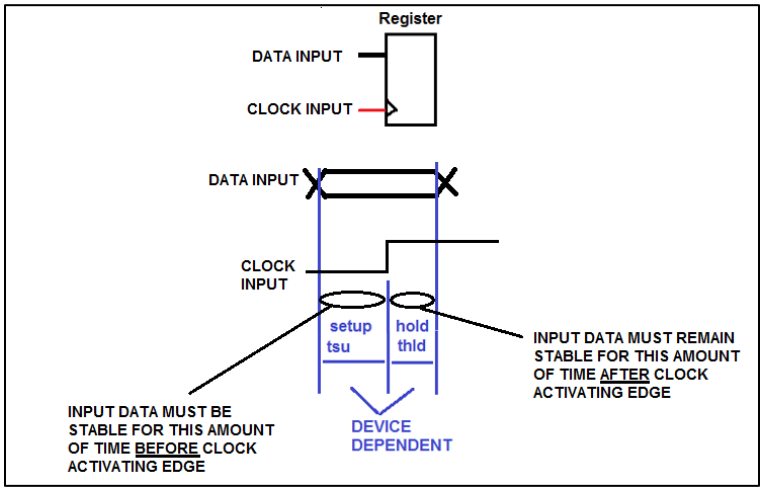

```
module ff_top (
input        data, enable,
input        clock,
output       outp
);

reg register;

always@(posedge (clock))
begin

if (enable == 1'b1)

register <=  data;

end

assign outp = register;
endmodule
```




```
module ff_top (
input        data, enable,
input        clock,
output [7:0] outp
);

reg [7:0] register;

always@(posedge (clock))
begin

if (enable == 1'b1)

register[7:0] <= {register[6:0], data};

end

assign outp = register;
endmodule
```

1.3.3 What is Metastability?

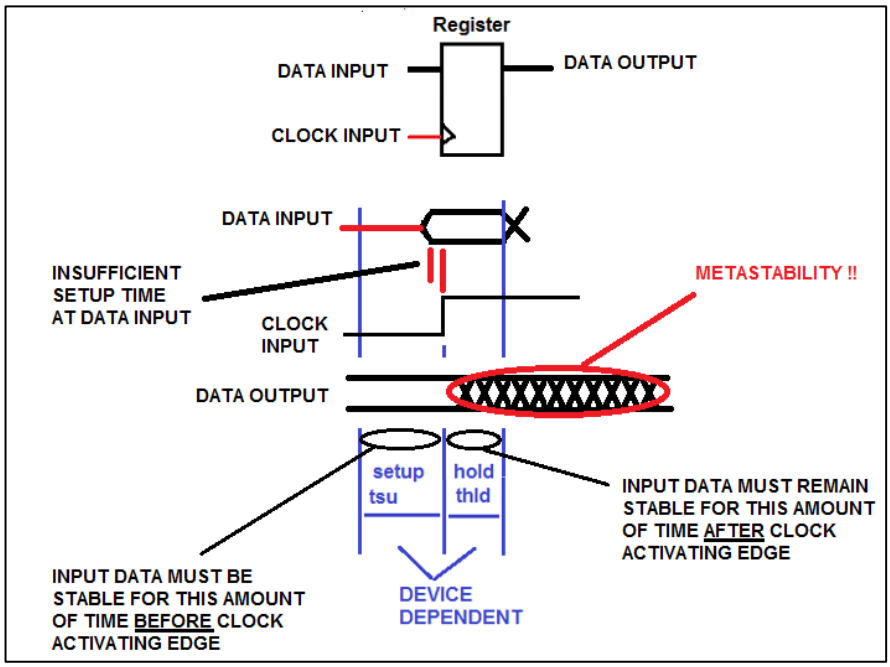
Every register has some basic timing constraints at its Data input known as “Setup time (tsu)” and “Hold time (thld)”. These constraints are expressed as relative to the activating edge of the register clock input. Refer to the figure below.



The Setup time (tsu) is the amount of time required for the data to be stable at the register input BEFORE the clock latching edge.

The Hold time (thld) is the amount of time required for the data to be stable at the register input AFTER the clock latching edge.

Violating these constraints can cause a problem at the register output. The figure below shows what happens to a register output when these timing constraints are not met. The “late arrival” of a Data Input relative to the register clock causes the Setup time violation. This result can be caused by an upstream signal delay due to signal routing in the device or by having too many levels of combinational logic to propagate through before the signal reaches the register.

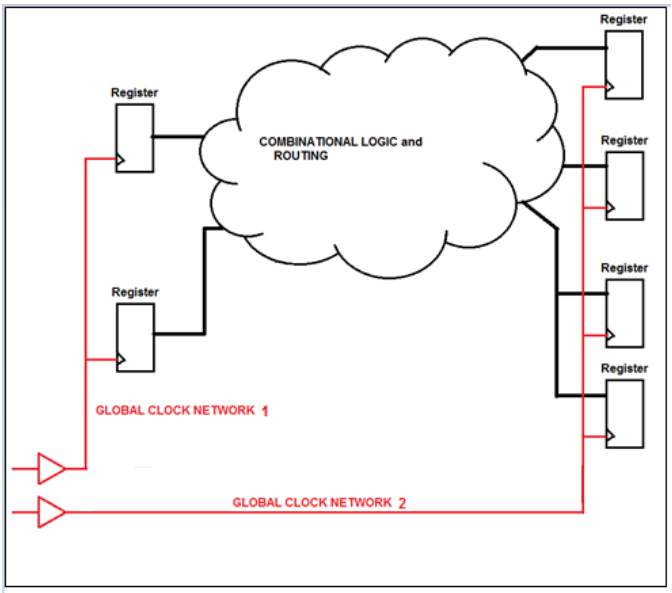


This situation at the output is called METASTABILITY and it can lead to all sorts of undefined and weird behaviours of downstream digital logic. The output drifts between the two logic levels for some amount of time. If other logic uses that output for logic operations, things can quickly get undefined.

Registers also have a timing property aligned with its output known as Clock to Output time”. The “tco” is the time delay between the clock arriving at a register and the changing of the output

of the register value. This aspect also has a role in the outcome of metastability issues with downstream registers.

But a BIG source of metastability problems in digital logic pipelines can come from employing multiple clock domains. These clock domains are asynchronous to each other (see below). Data outputs clocked registers from one clock domain may reach a pipeline register input that is running from a different clock domain.



Looking at the figure on the left, one can see that two register pipeline groups are connected (through the combinational logic and routing resources) but they run on different and ASYNCHRONOUS clock networks. For this situation the "robustness" of the design cannot be guaranteed. The operation is largely unpredictable.

There are techniques that allow the crossing of data and control signals between different clock domains but are beyond the scope of the course currently. For this course just one global clock source is to be considered for common clocking.

1.3.4 Using the Synchronous Design Techniques to Eliminate Metastability:

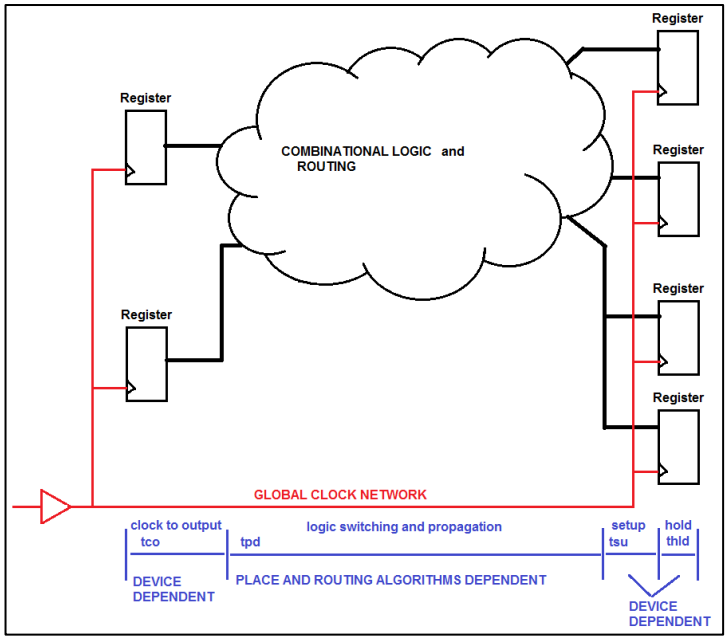
FPGA technologies have massive quantities of internal registers for state machines, counters, processors and all of them will use clock networking to run them. Between the registers, are equally huge numbers of combinational logic resources that do the digital logic transformations along the way.

To mitigate the possibility of METASTABILITY, an FPGA designer should take care in developing the arrangement of register pipeline designs. Use a common GLOBAL clock source as much as possible. This approach is usually referred to as "**Synchronous Design**".

In digital design work, there are several design constraints that we must consider in order to have the design be robust and operate with high performance reliability.

This Lab will employ the concept of "synchronous design" in mind. Metastability problems generally "show up" when synchronous design techniques are not used (i.e.: Asynchronous design). This can happen when pipeline registers are not using the same clock source. It might be good enough for VERY SIMPLE and SLOW circuits but is generally not acceptable for designs that will go into a production setting.

For all FPGA design situations, it is strongly recommended to follow a "best practices" approach to designing the clocking networks. This approach uses **Synchronous Design** clocking (shown in the figure below). This is done by transferring the data and control signals in the various register pipelines using a **COMMON**, high-performance (low skew) global clock network. This approach greatly reduces the range of operations variability. It makes the performance more predictable and easier to analyze.

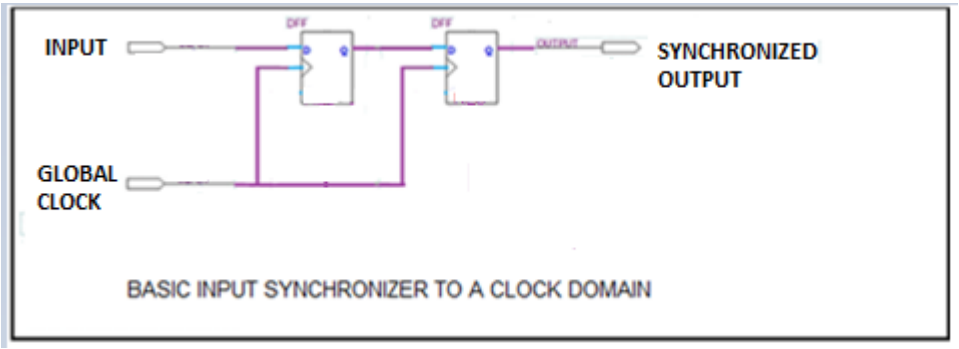


In synchronous designs, there is a Global clocking scheme used in a common clock domain. In the figure, one can see a typical array of registers used in a synchronous FPGA design. Between the SOURCE register on the left and the DESTINATION registers on the right, there can be a great deal of signal propagation delay along the paths through combinational logic and routing.

If it is deemed by the FPGA design tool, that moving data from one register output stage to another register input (on the same clock domain)) takes TOO LONG then the insertion of an intermediate clocked register pipeline stage is most likely required to be added.

However, someone may ask, "but external signals are NOT synchronized to this common global clock. They are ASYNCHRONOUS to the common global clock. How are these external signals supposed to be interfaced into a Synchronous Design?

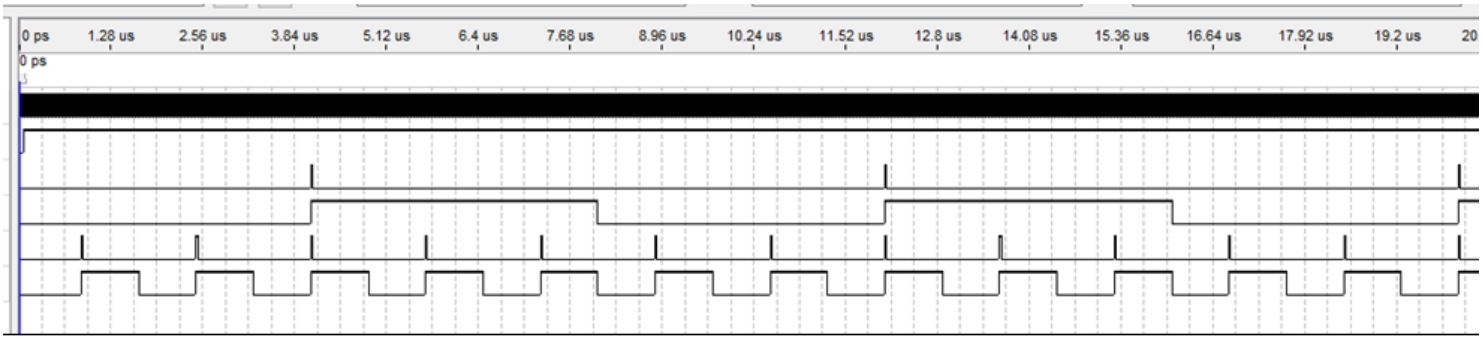
External Input Synchronization to the Global clock domain is done by adding two register stages in series (like a two-stage shift register) and clocked by the same common Global Clock.

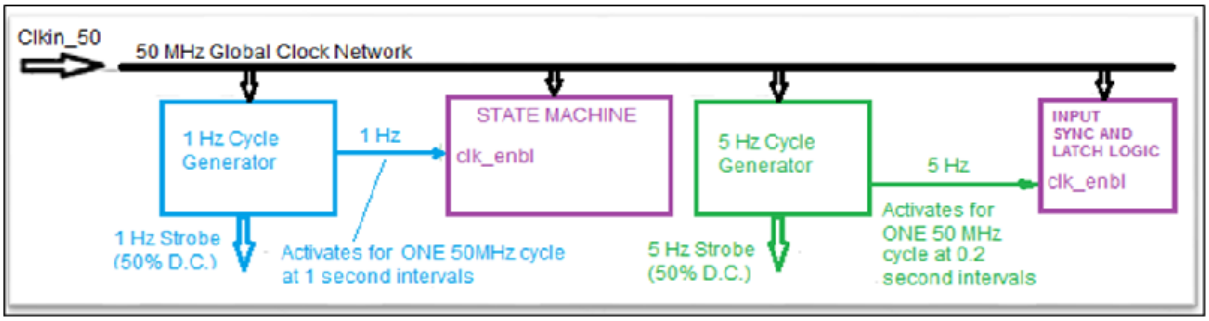


If metastability occurs at the output of the first register, the metastability will not reach the output of the second stage and thereby prevent it from getting into the rest of the digital logic.

Meanwhile the first stage output will have an entire Global clock cycle time for any metastable behavior to settle out before the second stage register transfers its input value from the first stage.

Below is a sketch of how a common global clock can be used to generate Clock_enables with different frequencies such that different logic functions can all run at different frequencies in a design. Each Clock_Enable must ALWAYS JUST BE ACTIVE FOR ONE CYCLE OF THE COMMON CLOCK SIGNAL.





1.3.5 Lab4 – New VERILOG Component – What is a State Machine?

Time-driven execution of any machine task can be accomplished by using a powerful design methodology. This approach employs STATE MACHINES.

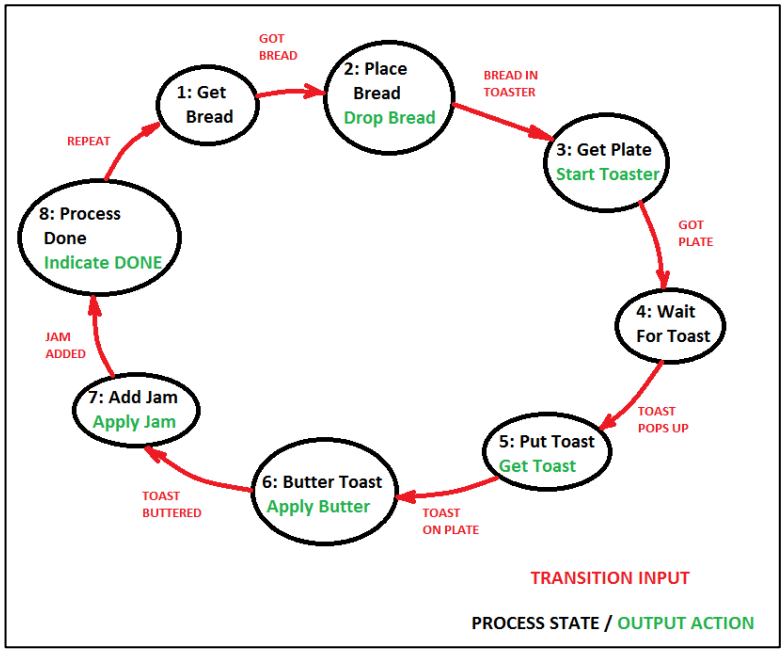
A trivial example task is itemized below for a Simple Breakfast outline:

STATE:	OUTPUTS:	TRANSITION EVENT TO NEXT
STATE:		
State 1: Get bread from cupboard		< Got Bread >
State 2: Placing bread in toaster	(DROP BREAD IN TOASTER)	< Bread in toaster>
State 3: Get plate	(START TOASTER)	< Got Plate >
State 4: Wait for Toast cycle completion		< Toast Pops Up >
State 5: Put Toast on plate	(GET TOAST FROM TOASTER)	< Toast on Plate >
State 6: Butter the Toast	(APPLY BUTTER)	< Toast Buttered >
State 7: Add Jam	(APPLY JAM)	< Jam Added >
State 8: Process done…..--> Enjoy.	(DONE)	<wait for REPEAT>

Observe the left-hand column that shows the name or "state" steps involved in the task. In the right-hand column are the transition inputs to the process that would be from sensors that could signify events such as "Got_Bread, Bread_in_Toaster_and_Started, Got_Plate" etc.

The logic outputs could be used to control the mechanics to "Drop Bread in Toaster", "Start Toaster",…."DONE" etc.

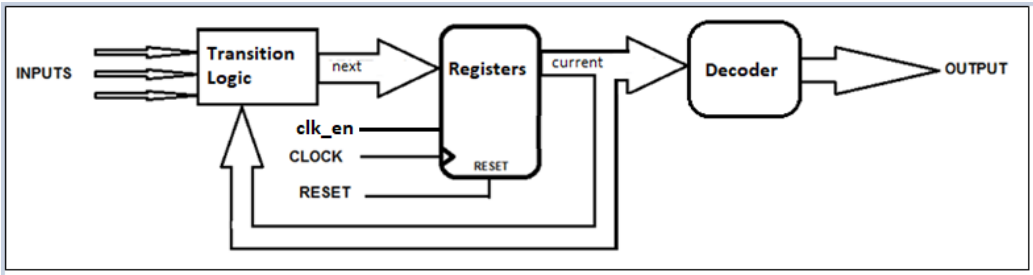
A state sequence diagram is shown below.



The task sequence could be implemented with a STATE MACHINE design to execute the sequence.

In a state machine design, the conditions that make it change from the CURRENT state to the NEXT state are various. Combinational logic is used to determine if-or-when those state transitions should occur. The state changes occur in synchronization with a state machine clock because registers are involved.

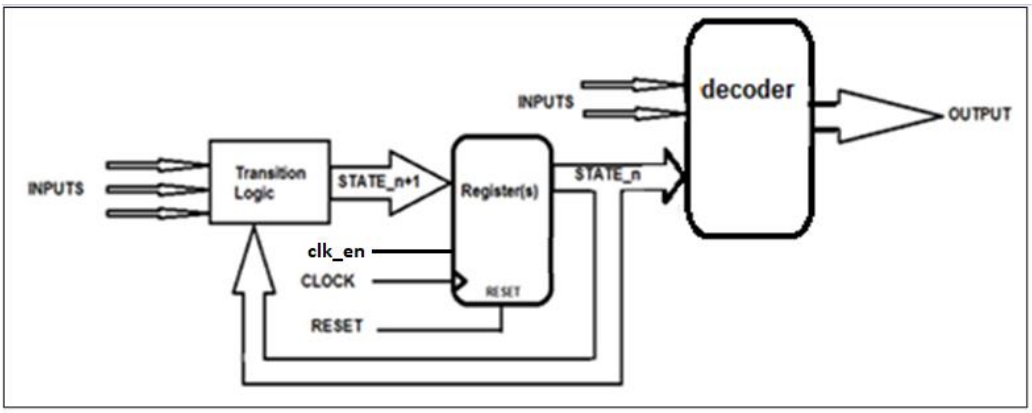
There are two primary classes of state machines used in sequential logic design: Moore and Mealy. To remember the names just think of two engineers sitting at a Thanksgiving table. One engineer politely says to the other "Would you like some Moore corn?" to which the second engineer replies, "Only if it's not Mealy". (Groan!).



MOORE STATE MACHINE:

Output is a function of present state ONLY.
Moore SM's outputs are only changed when the state machine is clocked.

For any change of an INPUT to a MOORE State Machine an output change will NOT happen until after the next CLOCK CYCLE.



MEALY STATE MACHINE:

Outputs are functions of present state AND inputs.
For any change of an INPUT to a MEALY State Machine an output change MAY happen with a ZERO delay or after the next clock cycle.

For this lab, both the Moore and Mealy state machines can be used. Referring to two figures above, there are generally just three sections in each state machine design. These are the Register, Transition and Decoder sections. Only the Register section is implemented as a Sequential Logic block. The Transition and Decoder sections are implemented as Combinational Logic blocks.

Now the three sections of the State Machine will be shown using the VERILOG always@ block construct.

REGISTER SECTION:

The state names must be defined first. In Verilog, the States must be represented with n-bit numeric values. They can be referred in the Register section by a any TEXT name. For example:

```
parameter       STATE_A   =3'b000,
                STATE_B   =3'b001,
                STATE_C   =3'b010,
                STATE_D   =3'b011,
                STATE_E   =3'b100,
                STATE_F   =3'b101;

reg [2:0] current_state, next_state;    // current_state, next_state registers for STATE CODES
```

The above states are declared as paramenters with names and unique numeric values. The number of bits used in the code must be able to at least cover the range of possible states being used by the state machine.

There are two variables used to contain any of the declared the State parameter codes. These are the “current_state” and the “next_state” variables

```
//REGISTER SECTION OF STATE MACHINE

always @(posedge clock)      // sequential logic to latch the FSM state (FSM REGISTER SECTION)

 begin
       if (reset)
          current_state <=  STATE_A;
       else if (sm_clken)
          current_state <= next_state;          // on the rising edge of clock the current state is updated with next state value
 end
```

The Register section uses the state machine clock input (**along with a clock_enable if specified**) and usually a RESET input signal. These signals are used to advance the state machine to a new state in when a clock edge (with a clock_enable) or to RESET it.

IT IS SEQUENTIAL LOGIC because **registers are created** (or inferred) with this always block construct.

TRANSITION SECTION:

The next State Machine section is the Transition section and it is implemented in another always block construct in the State Machine file. It is to be COMBINATIONAL LOGIC only. **THE ONLY OUTPUT FROM THE TRANSITION SECTION IS TO BE THE "next_state" value**. No State Machine ports to the outside world are allowed come from the State Machine Transition Section.
The always block uses an "*" in the Sensitivity list.
A VERILOG "CASE" statement is typically used.
Several State Machine inputs I0, I1, I2 and the "**current_state**" to evaluate what the "**next_state**" value will be. A typical example of a CASE statement is shown below:

```
//TRANSITION LOGIC SECTION

always@(*)
begin
 case (current_state)
        STATE_A:
              If (input1 == 1'b1)
                next_state = STATE_B;
              else
                next_state = STATE_A;

        STATE_B:
              if ((input1 == 1'b1) && (input2 == 1'b1))
                next_state = STATE_C;
              else
                next_state = STATE_B;

        STATE_C:
              if ((input1 ==1'b1) && (input3 == 1'b0))
                next_state = STATE_E;
              else if ((input1 ==1'b1) && (input3 == 1'b1))
                next_state = STATE_E;
              else
                next_state = STATE_C;

        STATE_D:
              if ((input1== 1'b1) && (input3 == 1'b1))
                next_state = STATE_E;
             else
                next_state = STATE_D;

        STATE_E:
              if ((input1 == 1'b0) && (input2 == 1'b0))
                next_state = STATE_B;
             else
                next_state = STATE_E;

        default:
                next_state = STATE_A;
 endcase
end
```

It MUST BE COMBINATIONAL LOGIC for proper State Machine operation and NO inferred latches are permitted.
NOTE: All IF statements must have ELSE CLAUSES PRESENT etc.)

Note how specific inputs are observed by the Transition Logic only in specific "current" state cases and also how these inputs can cause the State Machine Transition section to generate the "next"state values.

For example, if an "Input1" input signal is active when the State Machine "current_state" is in "STATE_A", the State Machine Transition section will force a jump in its state sequence to state "STATE_B" as its "next" state.

Some situations will result in no change in the next_state value.

A default state must always be included to cover any undefined parameter code values. If this is left out then latches may be inferred.

DECODER SECTION (Moore State Machine):

```
// MOORE Decoder Logic Section

always@(*)          // logic for decoder outputs from state code number

 begin
  case (current_state)
   STATE_A:
                begin
                output1     = 1'b0;
                output2     = 1'b0;
                 end
   STATE_B:
                begin
                output1     = 1'b0;
                output2     = 1'b1;
                end
   STATE_C:
                begin
                output1     = 1'b1;
                output2     = 1'b1;
                end
   STATE_D:
                begin
                output1     = 1'b1;
                output2     = 1'b0;
                 end
   STATE_E:
                begin
                output1     = 1'b0;
                output2     = 1'b0;
                 end
default:
                begin
                output1     = 1'b0;
                output2     = 1'b0;
                end
 endcase
end
```

A third section of the State Machine design is the Decoder section.

It sets the State Machine outputs for the entire State Machine.

For a Moore Style State Machine, the output values are determined only decoded from the value of the “current_state”.

It is a requirement that all the output signal levels are defined for all current_state values.

A default state must always be included to cover any undefined parameter code values. If this is left out then latches may be inferred.

The Decoder section is COMBINATIONAL LOGIC for proper State Machine operation and NO inferred are permitted.

DECODER SECTION (Mealy State Machine):

For a Mealy State Machine, the only major difference from the Moore State Machine is that the Decoder Section of a Mealy State Machine can have outputs that depend on the current_state value **AND** some inputs.

The Mealy State Machine state is like a "gate control" on an input signal (or a Boolean function of inputs) for an output activation.

Let's assume that the Transition Logic being the same as in the Moore example earlier.

```
always@(*)        // logic for decoder outputs from state code number
 begin
  case (current_state)
   STATE_A:
                  begin
                  output1 = 1'b0;
                  output2 = 1'b0;
                   end
   STATE_B:
                  begin
                    if (input1 == 1'b1)
                          begin
                           output1 = 1'b0;
                           output2 = 1'b1;
                          end
                     else
                          begin
                           output1 = 1'b0;
                           output2  = 1'b0;
                          end
                  end
   STATE_C:
                  begin
                  output1 = 1'b1;
                  output2 = 1'b1;
                  end
   STATE_D:
                  begin
                  output1 = input1;
                  output2 = input2;
                   end
   STATE_E:
                  begin
                  output1 = 1'b0;
                  output2 = 1'b0;
                   end
default:
                  begin
                  output1 = 1'b0;
                  output2 = 1'b0;
                  end
 endcase
end
```

Looking at the "output2" signal, it is driven ON when:

1) the current_state = STATE_B" AND if the input "input1" being ON or
2) when the current_state = STATE_C or
3) when current_state = STATE_D and input2 is ON.

Notice that for Mealy State machines (with the outputs also being driven by combinational inputs), the outputs can suffer from asynchronous transitions of the inputs. For example, if Input1 is "intermittent" then output1 and output2 will be "intermittent" as well during the enabling states. This is the main disadvantage of Mealy State Machines as compared to Moore State Machines.

The advantage for Mealy State Machines is that often they can be designed with a fewer number of states than a Moore version.

The external inputs are prevented from directly influencing the State Machine outputs. The Moore State Machine must change "state" first and then the outputs are driven by decoding the "current_state" value.

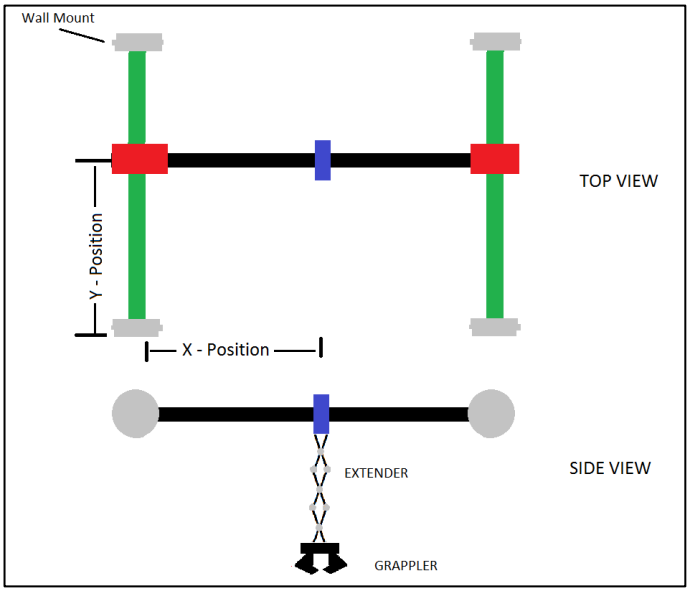
The type of state machine to be used for an application, depends on the requirements of the application.

The code examples above give you a starting point for a typical State Machine design. You can make the modifications to make your state machine operate as a Moore or Mealy design. You can declare it as a component at the top level and then create an instance of it to control other functions (shift register, counter etc.) and hook up the clock, inputs and outputs.

1.3.6 Lab4 Project Brief for Lab4

The Lab4 Project can encompass most of the components that you have designed earlier this term and in addition to the components that were developed earlier in this lab session.

You will create a Robotic Arm Controller or RAC (illustrated below) that could be used for positioning a robotic arm in 2 dimensions and employing an extender/grappler. Some requirements information on the operation of the RAC is needed.



RAC Movement Operations:The RAC movement operates on a basis of processing X/Y Target co-ordinates. The Target X/Y co-ordinate values are set by two sets of four switches (one set for X_target, one set for Y-target). The Target X and Y values will be captured and processed when a separate "MOTION" Push button is pressed and released.

When the MOTION button is **pressed** - the X/Y switch values must be CAPTURED into X and Y holding registers (i.e., Xreg and Yreg).

When the MOTION button is **released** - the RAC is enabled to move towards the X/Y Target. The motion must happen automatically, and it must continue until the RAC reaches that Target position. If the RAC reaches its Target X POSition before the Target Y-POSition, then motion in the X-direction is discontinued but motion in the Y-direction continues until the Target Y-position is reached. During this time of motion, further changes to the Target input values or the pressing of the Motion button must be prevented from updating the holding registers. The Target co-ordinates are to be captured into registers when the MOTION signal is pressed.

When the RAC is NOT in motion, the Extender can be enabled for operation.

NOTE: as mentioned earlier, X/Y motion of the RAC cannot happen if the Extender is in its extended position. If motion is attempted while the Extender is extended the RAC must generate a System Fault Error and X/Y motion is to be prevented. To clear the error the Extender must be fully retracted. Then a status indicator "Extender_OUT"must be updated to OFF. After this X/Y motion is permitted.

1.3.6.1 RAC Extender and Grappler Operation*:*

The Extender can be enabled to extend or retract when the RAC is not in motion. The Extender is activated by the RAC being stopped in X/Y motion and by pressing and releasing a dedicated "EXTENDER" push-button. **Pressing/holding/releasing** it once will cause the Extender to extend outwards and it should continue until the FULL extension is reached. **Pressing/holding/releasing** the button again (only when in the FULL EXTENDED position) will cause the retraction inwards and it should continue until the FULL RETRACTION is reached. The Extender position sequence is to be displayed on some leds(5 downto 2) as shown below:

Position:	leds[5:2]
Retracted:	0000
Extending1:	1000
Extending2:	1100
Extending3:	1110
Fully Extended:	1111

Anytime that the extender is NOT in its retracted position ("0000") the status flag "Extender_OUT" must be driven ON.

NOTE: If any attempt is made to move the RAC to new Target co-ordinates while the Extender_Out flag is active the X/Y Position Controller must be PREVENTED from X or Y motion and a System Fault Error must be indicated. The System Fault Error must remain active (locked) until the Extender is fully retracted.

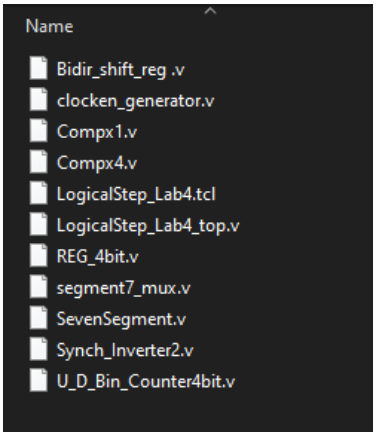
The Grappler is enabled to **change only when the Extender is in the Fully Extended position ("1111").** Its operation is enabled by a dedicated "GRAPPLER" push-button. **Press/Hold/Release** the button for an Open operation and **Press/Hold Release** the button for a Close operation. **The state of the Grappler must be indicated (1 means CLOSED; 0 means OPEN).**

ONLY ONE FUNCTION (ie: only ONE of Motion or Extender or Grappler) IS REQUESTED AT A TIME BY THE PUSH-BUTTONS. This simplifies the State Machine State sequence design.

FOR EACH BUTTON function you must sense the function Activation, The State Machine then proceeds to a new state and then it must wait to sense the function Deactivation and then proceed with the rest of its state sequence.

1.3.7 Initial Project Setup for Lab4

Start the Lab4 project like what was done in earlier Labs. Go to LEARN and download the Lab4 Zipped folder "Lab4" into the ECE-124 folder on your C: Drive. Extract the contents to create the new Lab4 project folder and its contents therein.

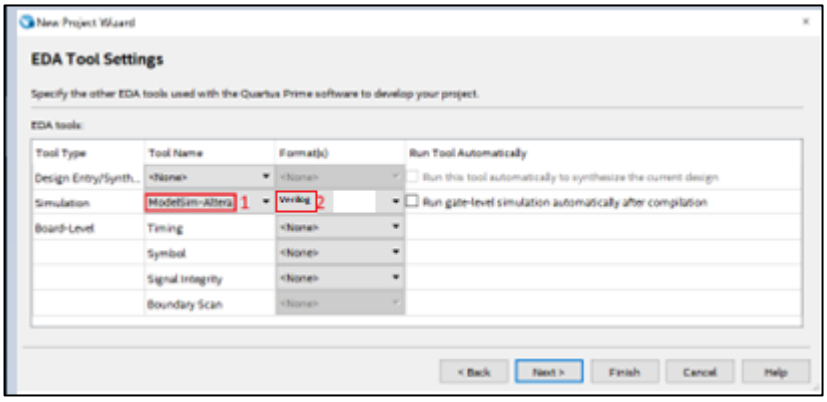


Your Lab4 Project Folder (downloaded from Learn) starts out like that shown at left. The files that will need to be developed are the LogicalStep_Lab4_top.v file and the THREE State Machine design files. For those three files, you must add ports to connect to the **Global_clock**, **Global_clken** and the **reset** signals in the LogicalStep_Lab4_top.v file, The LogicalStep_Lab4_top.v file already has these connections as inputs for most of the non-State-Machine Verilog blocks (Clken_Generator and Sync_Inverter blocks excluded).

Start up the Intel Quartus Prime platform and begin a new project with the New Project Wizard. Enter the new Project parameters:

Project Folder: Lab4
Project Name: LogicalStep_Lab4
Project Top Level: LogicalStep_Lab4_top

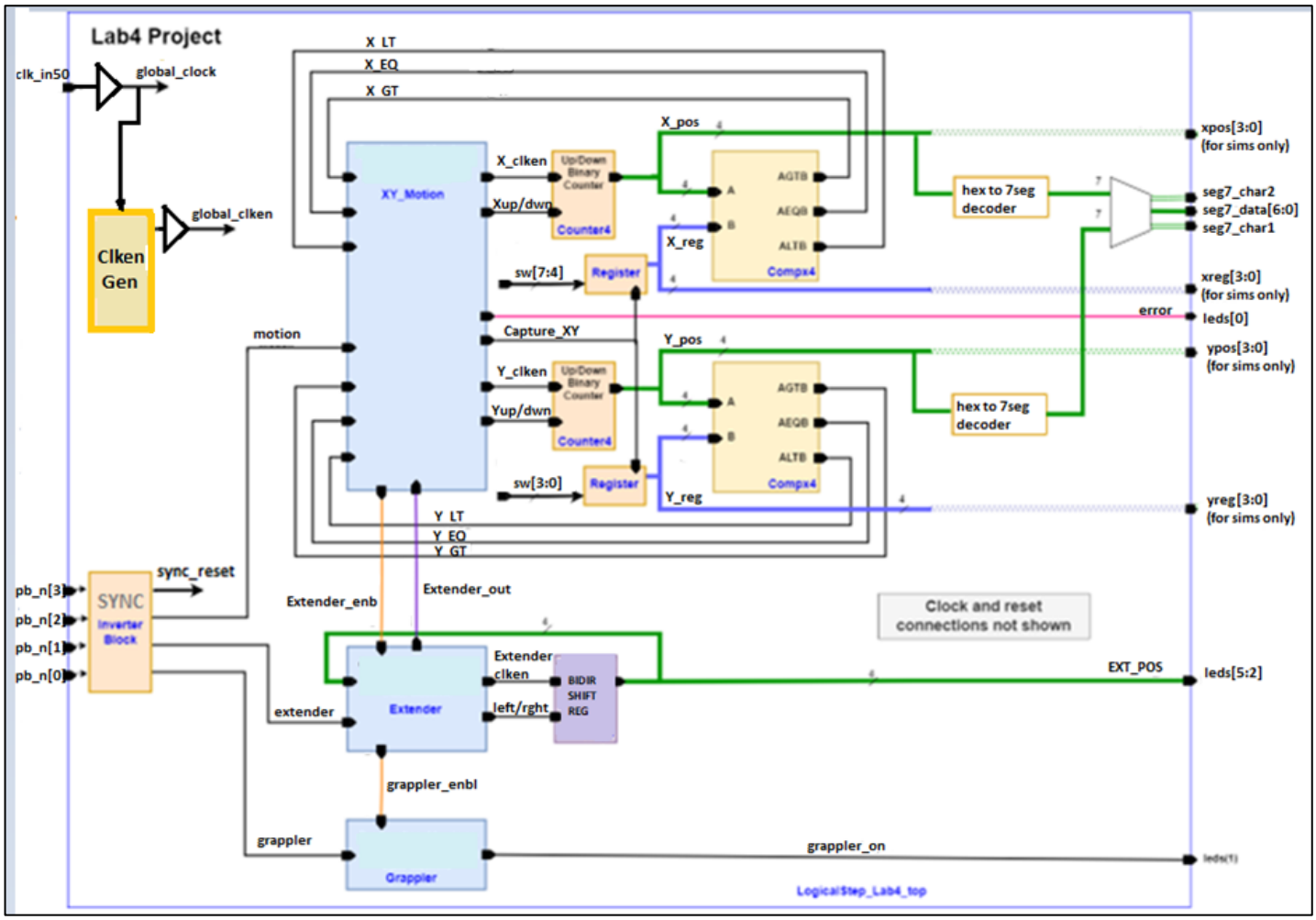
Then click NEXT on the New Project Wizard Dialog Window. Then click NEXT again (with Empty Project). Click NEXT on dialog windows for Family, Device & Board Settings until you reach the following window (EDA Tool Settings).



Click FINISH on the New Project Wizard Dialog Window.

NEXT IMPORTANT STEP (.tcl script):
Run the Lab4 TCL script to assign the FPGA device type and pins for the LogicalStep FPGA.

Below is a Block Diagram of the Lab4 Project FPGA design.



All Sequential Logic (registers, counters, and state machine register sections etc.) must be clocked by the signal "**global_clock**" and with the "**global_clken"** and the "**SYNC_RESET**" signals. Excepted from this are the SYNC_Inverter and Clken_Generator blocks.

The BLUE controlling blocks (XY_Motion/ Extender/Grappler) must each use a State Machine design. You must choose the Moore or Mealy State Machine for each controller block and make the connections to them.

To do your Lab4 project development, do one section at a time. Comment-out all sections except for the Infrastructure Section. Then just start by simulating the **clkin_50, global_clock, global_clken** and athe pb_n inputs and also the Sync Inverter outputs (**sync_reset, motion, extender and grappler**) signals first (from the Infrastructure Section). In the Simulator **Set all pb_n inputs to OFF and set the clkin_50 input to a 10ns clock period. Run the simulation and note the times when the global_clken is active.**

The State Machine states should have separate states to detect when the appropriate button is pressed and when the button is released. Only ONE button is pressed/held/released at a time.
The **motion, extender and grappler** signals must overlap the times when the global_clken is active to begin state machine operations. Leave the XY Motion Control and Extender Control sections commented-out.
Then develop the Grappler State Machine control first by creating the Grappler Control State Machine (SM2). You can force the grappler_enbl signal to ON and then activate the **grappler** input from the Sync Inverter block (using pb_n[0]).

Then move on the Extender State Machine control design and get the Bidirectional Shift register operating correctly. You can force the extender_enbl signal to ON and then activate the **extender** Input from the Sync Inverter block (using pb_n[1]).

Then move on to the X/Y Motion State Machine control etc.

1.3.7.1 Mealy State Machines (if used)

Any Mealy State Machine for the Lab4 Project will consist of 3 primary sections. These will be the Register, Transition and Decoder sections.

The Register section will be clocked by the rising edge of the Clk input. A reset input must also be included to clear the Register section back to its initial state.

The Transition section will allow the states to change states according to the inputs received from the inputs.

The Decoder section MUST be of the MEALY form. Outputs should always default to '0'. The outputs can otherwise be driven by inputs during specific current_state values.

1.3.7.2 Moore State Machines (if used)

Any Moore State Machine used in the Lab4 Project will consist of 3 primary sections. These will be the Register, Transition and Decoder sections.

The Register section will be clocked by the rising edge of the clock input. A reset input must also be included to clear the Register section back to its initial state.

The Transition section will allow the states to change states according to the inputs received from the inputs.
The Decoder section MUST be of the MOORE form. Outputs should be defined for each of the "current_state" values used and for the default case only.

The following snips show the Starter version sections of the LogicalStep_Lab4_top.v file

LogicalStep_Lab4_top.v Module Declaration:

```
`define SIM_FLAG

module  LogicalStep_Lab4_top (

   input clkin_50,
   input [3:0] pb_n,
   input [7:0] sw,
   output [7:0] leds,
`ifdef SIM_FLAG
   output [3:0] xreg, yreg,// (for SIMULATION only)
   output [3:0] xPOS, yPOS,// (for SIMULATION only)
   output       x_cnt_en_out,
   output       x_comp_gt_out,
   output       x_lt_out,
   output       y_cnt_en_out,
   output       y_comp_gt_out,
   output       y_comp_lt_out,
   output       clock_out,
   output       global_clken_out,
   output       clken_out,
   output       limit_out,
   output       motion_out,
   output       extender_out,
   output       grappler_out,
   output       extended_out,

`endif
   output [6:0] seg7_data, // 7-bit outputs to a 7-segment display (for LogicalStep only)
   output seg7_char1,      // seg7 digit1 selector (for LogicalStep only)
   output seg7_char2       // seg7 digit2 selector (for LogicalStep only)
   );
```

Like was done in the Lab3 project, the Lab4 project will use a set of Compiler Directives so that the development can be more time-efficient by observing the behaviour of internal signals in simulation. The list of signals in the **ifdef** section above may be changed to suit your needs. Most of these output ports are obvious in terms of the signal functions but you can refer to the Block Diagram earlier for clarification. Some signals are extras that you can delete. When the design is ready for testing, comment-out the define SIM_FLAG line above to remove the extra ports for the LogicalStep Board FPGA download file.

```
// outputs used for simulations only
`ifdef SIM_FLAG
   assign xreg                = x_capt_reg[3:0];
   assign yreg                = y_capt_reg[3:0];
   assign xPOS                = x_pos[3:0];
   assign yPOS                = y_pos[3:0];
   assign x_cnt_en_out        = x_cnt_en;
   assign x_comp_gt_out       = x_gt;
   assign x_comp_lt_out       = x_lt;
   assign y_cnt_en_out        = y_cnt_en;
   assign y_comp_gt_out       = y_gt;
   assign y_comp_lt_out       = y_lt;
   assign clock_out           = global_clk;
   assign global_clken_out    = global_clken;
   assign clken_out           = clken;
   assign limit_out           = limit_reached;
   assign motion_out          = motion;
   assign extender_out        = extender;
   assign grappler_out        = grappler;
   assign extended_out        = extended;
`endif
////////////////////////////////////////////////////

endmodule
```

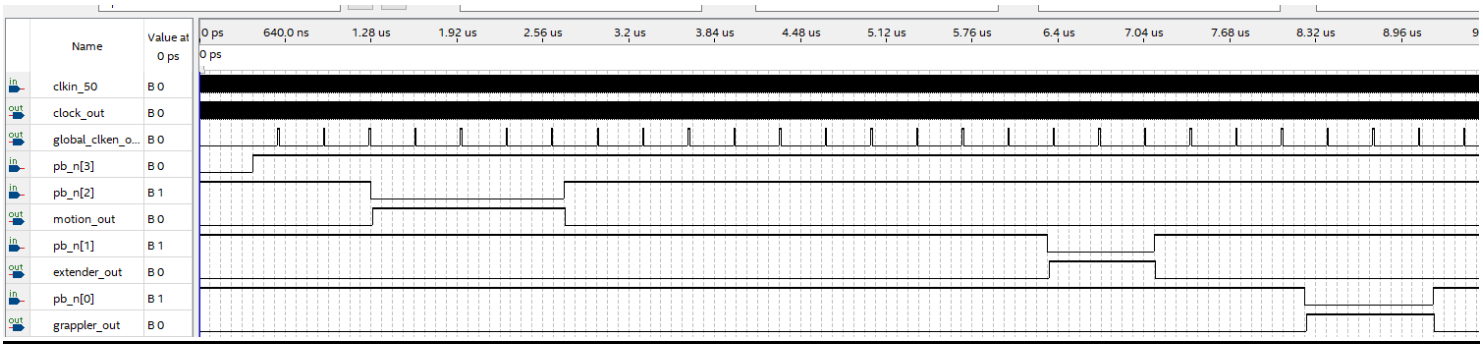
At the end of the file there are some signal assignments used to connect the extra output ports for simulation.

LogicalStep_Lab4_top.v file Infrastructure Section:

```
    ////////////////////////////////////////////////////////////////////////////////////////////////
    /////// START of INFRASTRUCTURE FUNCTIONS (CLOCK, CLOCK ENABLE AND SYNC SIGNALS) Section ///////

`ifdef SIM_FLAG
   localparam sim_flag = 1'b1;
`endif
`ifndef SIM_FLAG
   localparam sim_flag = 1'b0;
`endif

   wire     global_clk;    // Global Clock input used for all register clocking
   wire     clken, global_clken; // output used to periodicaLLY enAblE.
   wire     limit_reached;

clocken_generator #(.sim_flag (sim_flag)) U1 (
   .reset (reset),
   .global_clk (global_clk),        // Global Clock input used for all register clocking
   .limit_reached (limit_reached),
   .global_clken_source (clken)     // output used to periodicaLLY clock enablE registers.
);
   wire     reset, motion, extender, grappler;

   // invert the ACTIVE_LOW push-button inputs and then filter and synchronize them to the 50 MHz clock
Synch_Inverter2 U2 (
   .sync_clk (global_clk),
   .pb_n(pb_n),
   .sync_reset (reset),       // (pb_n(3)
   .sync_motion (motion),     // (pb_n(2)
   .sync_extender (extender), // (pb_n(1)
   .sync_grappler (grappler)  // (pb_n(0)
   );

// assign a GLOBAL BUFFER for the GLOBAL CLOCK (clkin_50)
   GLOBAL GLOBAL_CLOCK (.in(clkin_50), .out(global_clk));

// assign a GLOBAL BUFFER for the GLOBAL Clock Enable
   GLOBAL GLOBAL_CLKEN (.in(clken), .out(global_clken));

   ///////////////  END of INFRASTRUCTURE Section //////////////////////
   ////////////////////////////////////////////////////////////////////////////////////////////////
```

These blocks are pre-built for you. The **global_clken** signal (at the output the GLOBAL_CLKEN BUFFER changes to a high rate for simulation and a low rate for demo's via the SIM_FLAG macro (like in Lab3 for HVAC_SIM). The **limit_reached** output signal (you can ignore this signal) activates at half the strobe rate of the **global_clken** signal.

The Sync_Inverter2 block synchronizes the pb_n inputs to the 50 MHz global clock to drive the **sync_reset, motion, extender, grappler** signals. Inversion also happens to the pb_n signal inputs. Finally, this block does a "filtering" on the pb_n inputs to eliminate cross-ralk noise that sometimes occurs on the LogicalStep board. For the Motion, Extender, Grappler functions is simulation see below.



LogicalStep_Lab4_top.v Grappler Control Section:

```
    ////////////////////////////////////////////////////////////////////////
    ////////////////  START of GRAPPLER Controller Section /////////////////

    wire        grappler_enbl                   ; // Grappler activation enable
    wire        grappler_on                     ; // Grappler Status bits
// State Machine for Grappler Controller
    SM2 U5 (
    // LED outputs for Grappler status
    assign leds[1] = grappler_on;

    ////////////////  END of GRAPPLER Controller Section ////////////////////
    ////////////////////////////////////////////////////////////////////////
```

This Grappler Control Section is the simplest of the three blocks with a State Machine.

SM2 Inputs from the Infrastructure Section:

global_clk, global_clken, signals (coming from GLOBAL BUFFERS)
sync_reset, grappler signals (coming from the Sync Inverter2)

SM2 Inputs from the Extender Control Section:

grappler_enbl input signal (coming from the SM1 State Machine). During the initial development this signal may just be set to ON for simulations to bypass the Extender Control grappler_enbl signal.

SM2 Outputs:

The output signal will be the **grappler_ON** signal going the leds[1].
ON is Grappler Closed; OFF is Grappler Open

SM2 Final Operational Design:

The Grappler Control Section State Machine should enable the **grappler_ON** output signal to go through its OPEN or CLOSE sequence **only when the grappler_enbl** signal from the Extender Contral SM1 State Machine **is active AND** the **grappler** signal arrives from the Infrastructure Section

LogicalStep_Lab4_top.v Extender Control Section:

```
    ////////////////////////////////////////////////////////////////////////
    ////////////////  START of EXTENDER Controller Section /////////////////

    wire        extender_enbl,extender_in_motion;
    wire        extender_dir, extended          ; // Extender Control
    wire  [3:0] extender_pos                    ; // Extender extension position to be displayed on leds[7:4]

// State Machine for Extender Controller
    SM1_U4 (
    // Extender Shift Register
    Bidir_shift_reg_U15 (
    // LED outputs for Extender position status
    assign leds[5:2] = extender_pos[3:0];

    ////////////////  END of EXTENDER Controller Section ////////////////////
    ////////////////////////////////////////////////////////////////////////
```

This section is the next one of the three State Machine control blocks to be done.

SM1 Inputs from the Infrastructure Section:

global_clk, global_clken, signals (coming from GLOBAL BUFFERS)
sync_reset, extender signals (coming from the Sync Inverter2)

SM1 Inputs from the XY Motion Control Section:

extender_enbl input signal (coming from the SM1 State Machine). During the initial development this signal may just be set to ON for simulations to bypass the XY Motion Control extender_enbl signal.

SM1 Inputs from the Bidir Shift Register:

extender_pos[3:0] from the Bidir Shift Register to SM1 to sense the extender position. The Bidir Shift Register also connects the **extender_pos[3:0]** to leds[5:2]

SM1 Outputs:

extender_dir, extender_in_motion to the Bidir Shift Register for extend/retract
extended to the XY Motion Control section when the extender is **NOT FULLY RETRACTED**
grappler_enbl signal going the Grappler Control Section.

Final Operational Design:

This section will begin its sequence to extend or retract the Bidir Shift register (with **extender_dir, extender_in_motion**) **only when the extender_enbl** signal from the XY Motion Control State Machine (SM1) **is active AND** the **extender** signal arrives from the Infrastructure Section.
The Extender Control Section State Machine (SM1) should activate the **grappler_enbl** signal to the Grappler Contral Section **only if the extender is FULLY extended.**
The Extender Control Section should activate the **extended** signal to the XY Motion Control section (SM) **whenever the extender is NOT FULLY RETRACTED**.

LogicalStep_Lab4_top.v X/Y Motion Control Section:

```
    ////////////////////////////////////////////////////////////////////////////
    ////////////////START of X/Y Controller Section ///////////////////////////

    wire        x_eq, x_gt, x_lt;
    wire        x_cnt_en, x_cnt_up1_dwn0      ;
    wire  [3:0] x_pos                         ; // X position
    wire  [3:0] x_target                      ; // X target value from switches
    wire  [3:0] x_capt_reg                    ; // X target capture register

    wire        y_eq, y_gt, y_lt;
    wire        y_cnt_en, y_cnt_up1_dwn0      ;
    wire  [3:0] y_pos                         ; // y position
    wire  [3:0] y_target                      ; // y target value from switches
    wire  [3:0] y_capt_reg                    ; // y target capture register

    wire        capture_enable                ; // Target Co-ordinate Capture enable signal
    wire        posc_err                      ; // Position Controller Error status bit

    wire  [6:0] seg7_x                        ; // signals for x-position
    wire  [6:0] seg7_y                        ; // signals for y-position

// State Machine for X/Y Position Controller
    SM_U3 (
    assign x_target = sw[7:4];
// X-Co-ordinate Position capture register
    REG_4bit U6 (
    assign y_target = sw[3:0];
// X-Co-ordinate Position capture register
    REG_4bit U7 (
// X-Co-ordinate Position counter
    U_D_Bin_Counter4bit U8 (
// Y-Co-ordinate Position counter
    U_D_Bin_Counter4bit U9 (
// hex comparators for x/y position values
    Compx4 U10 (
    Compx4 U11 (
// Y Position decoder
    SevenSegment U12 (
// Y Position decoder
    SevenSegment U13 (
// X-Y Position displays
    segment7_mux U14 (
// LED outputs for MOTION ERROR (LOCKED) status
    assign leds[0] = posc_err;

    ////////////////  END of X/Y Controller Section ////////////////////////
    ////////////////////////////////////////////////////////////////////////////
```

SM Inputs from the Infrastructure Section:

global_clk, global_clken, signals (coming from GLOBAL BUFFERS)
sync_reset, motion signals (coming from the Sync Inverter2)

SM Inputs from the XY Motion Control Section:

x_gt, xeq, x_lt come from the X-Compx comparator
y_gt, yeq, y_lt come from the Y-Compx comparator

SM Inputs from the Extender Control Section:

extended from the SM1 State Machine

SM Outputs:

Capture_XY goes to the X and Y REG_4bit registers to capture the Target Co-ordinates (as a synchronous load (like a clk_enable) when the Motion input arrives.
x_cnt_en is activated to allow the x-position counter to count.
x_cnt_up1_dwn0 is used to control the x_position counter counting direction.
y_cnt_en is activated to allow the y-position counter to count.
y_cnt_up1_dwn0 is used to control the y_position counter counting direction.

extender_enbl going to the Extender Section State Machine SM1.

posc_err goes to leds[0] and is active when the XY motion is requested but the **extended** signal is active.

Final Operational Design:

Reset is activated and the seven-segment patterns go to 00 on the Dual seven-segment display.
The X and Y Target co-ordinates are set with sw[4], sw[3:0] respectively.
The pb_n[2] is pressed and HELD for 1 second and then released. The XY Motion controller should start moving towards the X/Y target co-ordinates and the actual position is shown on the Dual Seven-segment display (Digit1 shows X, Digit2 shows Y). They must both change together unless one target is reached first (then the other continues towards its target). While in motion, changing the Target c-ordinate values with the switches must not be allowed override the current target destination.
When finally reaching the target, the XY Motion becomes “AT-REST and a new target co-ordinate set may be processed by changing the XY Target value and activating the pb_n[2] button again.
Anytime that the XY Motion is requested and the **extended** signal (from the Extender Control Section) is active, the SM XY Motion Control will activate the **posc_err** output. This may be cleared by retracting the extender and then press/HOLD/release the pb_n[2] button for the Motion operation again.

Lab4 Project Input / Output Definitions

SIGNAL TYPE:	SIGNAL NAME:	ASSIGNED PORT(s):	Comment
RAC Inputs	Clock	Clk	Main clock driven by clock source block
	RESET	pb_n[3] – inverted, filtered, sync'd	RESET – must reset ALL registers and State Machines
	X_target	sw[7:4]	Target X- Co-ordinate (in hex)
	Y_target	sw[3:0]	Target Y- Co-ordinate (in hex)
	Motion	pb_n[2]- inverted, filtered, sync'd	Capture X/Y and enable Motion to X/Y Target – press and hold for 1 second and then release to start motion
	Extender	pb_n[1]- inverted, filtered, sync'd	Extender Toggle (press and hold for 1 second and then release to extend; press and hold for 1 second and then release to retract)
	Grappler	pb_n[0]- inverted, filtered, sync'd	Grappler Toggle (press and hold for 1 second and release to open; press and hold for 1 second and release to close)
RAC Outputs	extender_position	leds[5:2]	Shows extender position
	"diagnostics"	leds[7:6]	SPARE LEDs: Use to connect to internal signals for debugging if needed
	grappler_on	leds[1]	Shows Grappler open/close status (1 means CLOSED; 0 means OPEN)
	Posc_err	leds[0]	Shows System Fault Error
LogicalStep Board Outputs (not for sims)		Seg7_char2	Enables the seg7_data for DIGIT2 display
	Seg7_data	Seg7_data[6:0]	Segment7 data
		Seg7_char1	Enables the seg7_data for DIGIT1 display